arXiv:2405.14906v1 [cs.SE] 23 May 2024

AutoCoder: Enhancing Code Large Language Model with AIEV-INSTRUCT

Bin Lei[1] **Yuchen Li**[2] **Qiuwu Chen**[2]
[1]University of Connecticut [2]AIGCode
{bin.lei}@uconn.edu {liyuchen, chenqiuwu}@aigcode.net

Abstract

We introduce AutoCoder, the first Large Language Model to surpass GPT-4 Turbo (April 2024) and GPT-4o in pass@1 on the Human Eval benchmark test ($\mathbf{90.9}\%$ vs. $\mathbf{90.2}\%$). In addition, AutoCoder offers a more versatile code interpreter compared to GPT-4 Turbo and GPT-4o. It's code interpreter can install external packages instead of limiting to built-in packages. AutoCoder's training data is a multi-turn dialogue dataset created by a system combining agent interaction and external code execution verification, a method we term **AIEV-INSTRUCT** (Instruction Tuning with Agent-Interaction and Execution-Verified). Compared to previous large-scale code dataset generation methods, AIEV-INSTRUCT reduces dependence on proprietary large models and provides execution-validated code dataset. The code and the demo video is available in https://github.com/bin123apple/AutoCoder.

1 Introduction

Code generation is a critical aspect of modern software development. It significantly enhances development efficiency and quality by increasing productivity, reducing errors, standardizing code, accelerating prototyping, and supporting complex systems [9, 21, 22].Recently, Large Language Models (LLMs), such as GPT-4 [26] and CodeQwen1.5 [29], have achieved significant advancements on code generation. These models have shown high accuracy in producing code that meets user requirements and have been widely adopted in real-world software development.

Training large language models requires extensive high-quality data [17]. This is particularly crucial for code generation tasks that demand high accuracy [13]. OpenAI once hired people to help annotate the Code Instruct dataset for training their InstructGPT [28]. However, manually annotating large-scale code instruction datasets is both economically and time-consuming [33]. To address this challenge, previous work has employed various automated code annotation methods, such as SELF-INSTRUCT [31], EVOL-INSTRUCT [25], and OSS-INSTRUCT [32]. SELF-INSTRUCT enhances LLMs' instruction-following capabilities by using strong teacher models to generate synthetic coding instructions for fine-tuning weaker student models. EVOL-INSTRUCT improves LLMs' coding abilities by iteratively increasing the complexity of seed code instructions through various heuristics. OSS-INSTRUCT generates diverse and realistic coding problems by drawing inspiration from open-source code snippets. The essence of these methods lies in distilling the knowledge of a powerful teacher model (such as GPT-4 Turbo) to guide a smaller model. This leads to a problem: **While the small model can achieve significant performance improvements, the final accuracy of the small model is unlikely to surpass that of the teacher model.** Because both the correct and incorrect knowledge from the teacher model are transferred to the small model. Moreover, although using closed-source models reduces costs compared to manual annotation, the cost of using closed-source models remains high. According to our tests, even with the relatively cheaper GPT-4 Turbo model, generating an average of 6,500 high-quality entries for the code instruction dataset costs $1,000.

Preprint. Under review.

This raises two questions:

1. ***Can we correct the incorrect knowledge generated by the teacher model to provide more accurate code for the student model?***
2. ***Instead of relying on expensive closed-source teacher models, can we enable our student model to learn autonomously?***

To address the **first issue**, we designed a new large-scale code instruction dataset annotation method called AIEV-INSTRUCT. It is an interaction system comprising two agents: a ***questioner*** and a ***programmer***. These agents interact to simulate the process of ***programmers*** constructing code according to project requirements and conducting unit tests. In each dialogue round, we extract the code generated by the ***programmers*** and execute it. The execution results are returned to the ***questioner*** to inform the next round of questions. This process continues until the ***programmers***'s code passes the unit tests, ensuring the accuracy of the generated code dataset.

To address the **second issue**, we sperate AIEV-INSTRUCT into two stages: the *Teaching Stage* and the *Self-learning Stage*. In the *Teaching Stage*, we rely on proprietary large models as agents for code annotation, similar to previous methods. Once our model surpasses the proprietary models in accuracy on the test set, we transition to the *Self-learning Stage*. In this stage, we use our own model as the agent for code annotation. For detailed methodology, refer to Section 3.

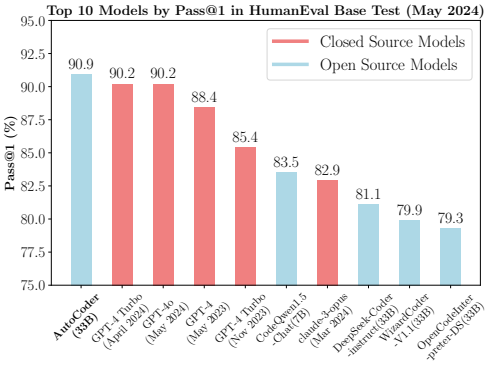

Figure 1: Pass@1 (%) comparison of Various LLMs on the HumanEval Base Test.

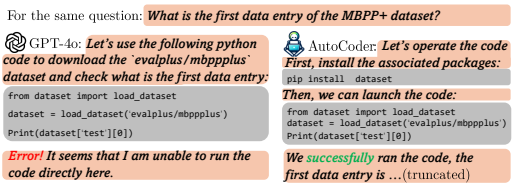

Figure 2: Comparison of Code Interpreter Functions between AutoCoder and GPT-4o. : Nature language generated by the model; : Code generated by the model. AutoCoder can recognize **external package** installation commands, whereas GPT-4o can only run code that includes built-in packages. The demo video is in https://github.com/bin123apple/AutoCoder .

Under the support of AIEV-INSTRUCT, we obtained 169K high-quality code instruction data samples. Using this dataset, we trained the AutoCoder series models, including AutoCoder (33B) and AutoCoder-S (6.7B). As shown in Figure 1, AutoCoder demonstrates higher accuracy. In the HumanEval Base Test, we compared it with the top ten models on the current (May 2024) EvalPlus Leaderboard [3]. The results indicate that AutoCoder's Pass@1 even surpasses that of the current top-ranked models, GPT-4 Turbo (April 2024) and GPT-4o.

Moreover, as illustrated in Figure 2, AutoCoder boasts a more versatile Code Interpreter function compared to GPT-4o and GPT-4 Turbo. The Code Interpreter is an external program execution environment that large models utilize to execute the code they deem necessary. While GPT-4o and GPT-4 Turbo can identify the code that needs to be executed, they fail to provide the Code Interpreter with the necessary instructions to install external packages required by the programs. This limitation significantly restricts the capabilities of the Code Interpreter. In contrast, AutoCoder can correctly supply the Code Interpreter with the appropriate external package installation instructions, thereby enabling it to execute a wide variety of code.

To comprehensively evaluate the capabilities of AutoCoder, we tested it on several datasets: HumanEval [13], HumanEval+ [24], MBPP [8], MBPP+ [24], MultiPL-E [11], and DS-1000 [20]. To measure the performance improvement of AutoCoder, we compared it to its base model, Deepseek-Coder [16]. The results demonstrate that AutoCoder exhibits outstanding performance. As of May 2024, AutoCoder ranks **1st** among all LLMs on the HumanEval Base Test, **5th** on the HumanEval Plus Test, and **4th** on both the MBPP Base Test and the MBPP Plus Test. Detailed experimental procedures can be found in Section 5.

Overall, our contributions are summarized as follows:

We propose AIEV-INSTRUCT, a novel method for creating high-quality large code datasets. It simulates programmers writing code and conducting unit tests through agent interactions, ensuring annotation accuracy with an external code executor. It includes a *Teaching Stage* and a *Self-Learning Stage*, reducing reliance on expensive closed-source models during the annotation process.

We introduce AutoCoder, a code LLM trained using AIEV-INSTRUCT that excels in code-related tasks. It outperforms top models like GPT-4 Turbo and GPT-4o on the HumanEval benchmark.

We enhances the functionality of the current code interpreters. AutoCoder can provide the code interpreter with the necessary instructions to install external packages, extending the applicability of the code interpreter beyond built-in packages.

2 Related Work

Large Language Models for Code. Recently, LLMs have shown remarkable abilities in understanding and generating code [19]. Trained on extensive datasets covering various programming languages and tasks, these models excel in code completion, bug fixing, and code synthesis [18]. Closed-source models like OpenAI's GPT-4 [26], Claude.ai's Claude [1], and Google's Gemini [4] series have demonstrated superior performance on code tasks. Meanwhile, open-source models specialized for code, such as DeepSeek-Coder [2], CodeQwen [29], Magicoder [32], OpenCodeInterpreter [34], and WizardCoder [25], are also emerging. Generally, closed-source models outperform open-source ones due to their larger parameter sizes and broader knowledge base.

Code LLMs Instruction Tuning. After pre-training large models, we use instruction tuning to optimize them [15], enhancing their ability to understand and execute specific instructions [12]. A major challenge in Instruction Tuning for Code LLMs is the lack of high-quality instruction datasets for code [30]. Code tasks, such as Text-Code and Code-Code translation, are difficult and time-consuming to annotate manually. OpenAI used human annotators to label various tasks and train InstructGPT [28], but they noted that annotating code tasks is prohibitively expensive for large-scale datasets. Since the advent of GPT-4, an increasing number of researchers have leveraged GPT-4 for code annotation to create high-quality instruction tuning datasets. Currently, there are three primary methods: SELF-INSTRUCT[31], EVOL-INSTRUCT[25], and OSS-INSTRUCT [32]. SELF-INSTRUCT boosts LLMs' instruction-following skills by using strong teacher models to generate synthetic coding instructions for fine-tuning weaker student models. EVOL-INSTRUCT iteratively enhances LLMs' coding abilities by increasing the complexity of seed code instructions. OSS-INSTRUCT creates diverse coding problems inspired by open-source code snippets. These methods distill the expertise of powerful teacher models like GPT-4 to guide and improve smaller models.

3 AIEV-INSTRUCT

3.1 Overall Architecture

Figure 3 illustrates the overall architecture of AIEV-INSTRUCT, divided into two stages: the *Teaching Stage* and the *Self-Learning Stage*. In the *Teaching Stage*, the model learns primarily by distilling knowledge from a teacher model. In the *Self-Learning Stage*, it learns autonomously.

In the *Teaching Stage*, we obtain open-source code snippets and use GPT-4 Turbo as the teacher model to supplement and correct them. The process consists of four main steps. **In I : Initialization,** we initialize the necessary components. GPT-4 Turbo is assigned two roles: ***questioner*** and ***programmer***. It can ensure the generated data is diverse, resulting in a more uniform probability distribution rather than converging to a specific dialogue template. The dialogue messages are initialized as an empty list, which will be used throughout the process to store data. Eventually, this list will contain multiple rounds of dialogue, and the entire conversation will be added as a single data entry to our final dataset. Additionally, we need to initialize a Docker container as our Code Interpreter. This container is responsible for installing the required external packages and executing the code that needs verification throughout the process. **In II : Propose the question,** we first utilize GPT-4 Turbo to execute OSS-Instruct [32], designing a problem description and a specific solution that includes the code snippet based on the open-source code fragment. The difference here is that we require GPT-4 Turbo to provide some **Unit Tests**. These **Unit Tests** further ensure the accuracy of the code in

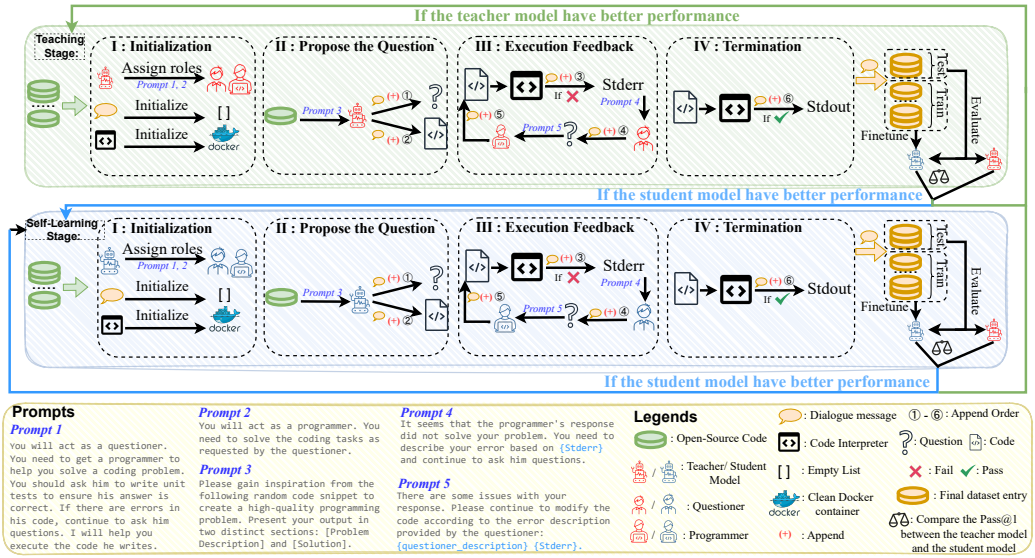

Figure 3: The overall architecture of the AIEV-INSTRUCT.

our dataset. The dialogue messages initialized in the previous step are sequentially appended with the problem description (①), the solution and the unit tests (②). **In III: Execution Feedback:**, we use multiple rounds of execution feedback to check the generated code, thereby improving the quality of the dataset. First, we input the code snippet generated in the second step into the Code Interpreter. If an execution error occurs, the dialogue messages append the detailed Stderr output (③). Meanwhile, this Stderr information is provided to the ***questioner***, who will generate a natural language description based on the Stderr. This natural language description is also appended to the dialogue messages (④). Next, both the natural language description and the Stderr are provided as new questions to the ***programmer***, who will continue to modify the code. The dialogue messages will append the new code it generates (⑤) and repeat this process. **In IV: Termination**, we also use the Code Interpreter to run the code generated by the ***programmer***. If the program executes successfully, the Stdout is appended to the dialogue messages (⑥). This completes the analysis of one data entry.

After analyzing every 2000 data entries, we split the new data into a test set and a training set in a 1:9 ratio. The training set is used to train the student model (AutoCoder). After training, we use the test set to evaluate both the teacher model and the student model. Upon completion of the evaluation, we compare the Pass@1 of the two models. If the teacher model performs better, we continue executing the *Teaching Stage*. If the student model performs better, we move to the *Self-Learning Stage*. The difference between the *Self-Learning Stage* and the *Teaching Stage* is that in the *Self-Learning Stage*, we replace the original teacher model with the student model. The student model itself is assigned as the ***questioner*** and ***programmer*** , and it completes the entire execution feedback process.

3.2 Dataset Analysis

Dataset Generation. To prevent **data contamination** in test sets from resulting in overly high performance on certain benchmark datasets (such as HumanEval), we used code from two datasets that had already undergone contamination detection: *Magicoder-Evol-Instruct* and *Magicoder-OSS-Instruct* [32]. We collected a total of 186K original code entries from these two datasets. After de-duplication, we input these data into our AIEV-Instruct pipeline to generate the dataset. We set the maximum number of execution feedback iterations in AIEV-Instruct to 7. If the generated code fails to execute successfully and pass all unit tests after 7 attempts, that data point is discarded. The `gpt-4-turbo-2024-04-09` is used as the teacher model.

Dataset Comparision. We compared our dataset *AutoCoder-AIEV-Instruct* with several current large code instruction datasets. The comparison results are shown in Figure 4. The dataset *AutoCoder-AIEV-Instruct* contains 169K data samples, totaling 241K rounds of dialogue. Among these, 150K rounds are contributed by multi-round dialogue data samples. Besides including the main function, it also encompasses subsequent package installations, code execution errors, or results, as well as various

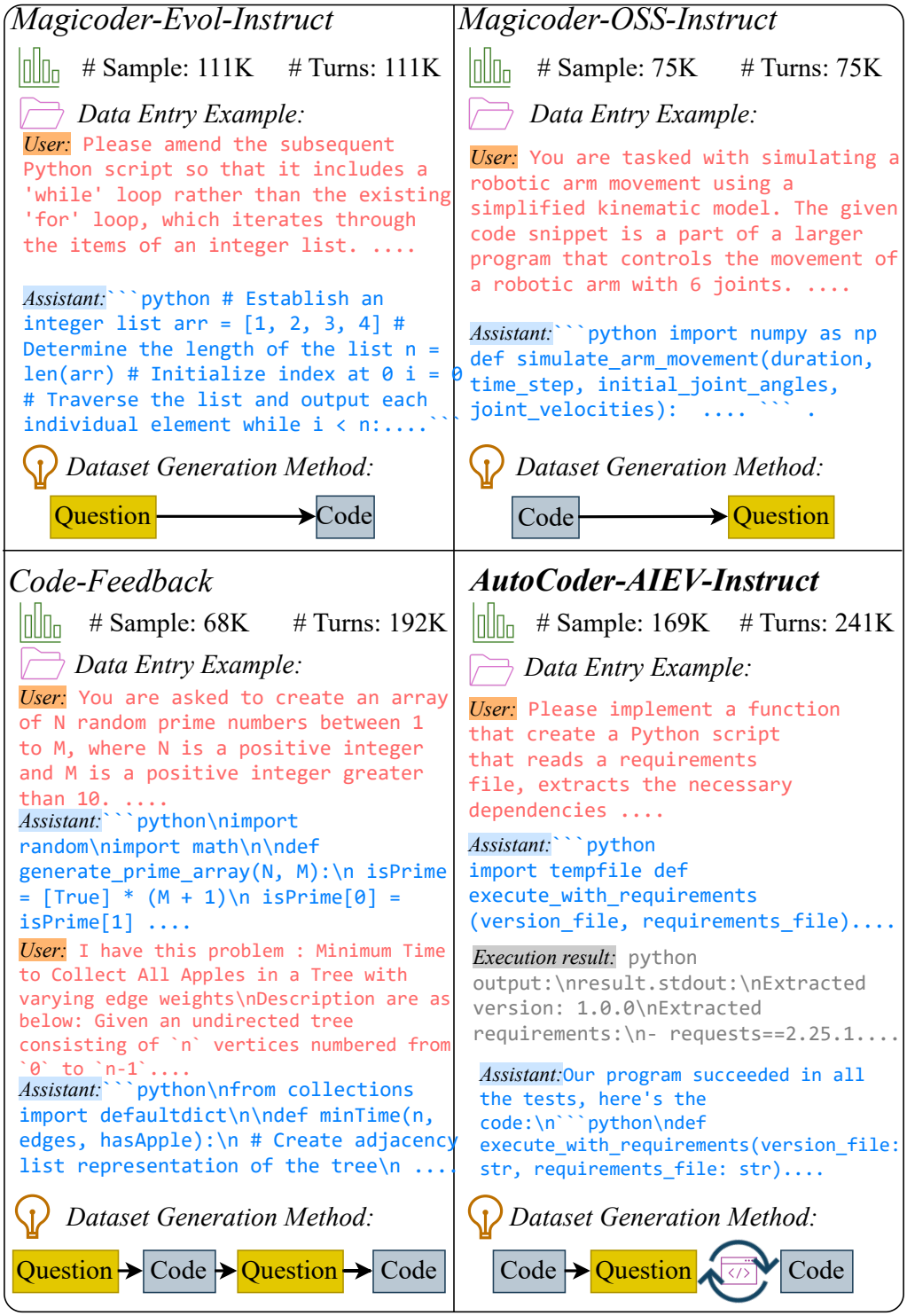

Figure 4: The comparison between the AutoCoder-AIEV-Instruct and other large code datasets.

error analyses. Compared to the original *Magicoder-Evol-Instruct* and *Magicoder-OSS-Instruct*, it adds unit tests, which further enhances the accuracy of code-related tasks. Additionally, compared to *Code-Feedback* [34], it includes more execution feedback results, reducing the multi-round dialogues for code block concatenation and enhancing the coherence of the context.

Dataset Decontamination. Similar to the data processing method used by StarCoder [23], we also performed decontamination for *AutoCoder-AIEV-Instruct*. Specifically, we tested each code snippet from HumanEval, MBPP, DS-1000, and MultiPL-E against every code snippet in *AutoCoder-AIEV-Instruct* using Levenshtein distance. If the similarity exceeded 90%, the data entry was removed. Through this process, we excluded a total of 113 data entries.

Dataset Accuracy Theoretical Analysis. EVOL-INSTRUCT generates code from questions using a teacher model. Thus, the theoretical maximum accuracy of the EVOL-INSTRUCT dataset should closely match the teacher model's accuracy in generating correct code c for given problems p. Thus, we get $\mathcal{A}_{Evol} \approx \mathcal{P}(c \mid p)$. OSS-INSTRUCT generates problems from open-source code using a teacher model. Therefore, the theoretical maximum accuracy of the OSS-INSTRUCT dataset should closely match the teacher model's accuracy in analyzing and interpreting open-source code c. Thus, we get $\mathcal{A}_{OSS} \approx \mathcal{P}(p \mid c)$. For AIEV-INSTRUCT, we align problem descriptions and code by asking LLMs to add unit tests to the original code. After adding unit tests, we can obtain new code c^* and new problem descriptions p^*. Here, we can make an assumption: $\mathcal{P}(p \mid c) < \mathcal{P}(p^* \mid c^*)$. This is because LLMs usually find it easier to align problem descriptions with unit tests. For example, suppose we need to write a program `is_prime` to detect prime numbers in a list. It is easy to validate the code by providing a few unit tests such as `assert is_prime([2,3,4]) == [2,3]` but ensuring that the corresponding program is entirely correct is not as straightforward. with iterative validation and correction, the probability of correctness improves with each iteration. If the probability of error in each iteration is $1 - \mathcal{P}(p^* \mid c^*)$, and assuming each iteration is independent, the probability of correctness after n iterations is: $\mathcal{A}_{AIEV} \approx 1 - (1 - \mathcal{P}(p^* \mid c^*))^n > 1 - (1 - \mathcal{P}(p \mid c))^n$.

While prior probability of problems $\mathcal{P}(p)$ is greater than the prior probability of code $\mathcal{P}(c)$ and the iteration times n is greater than 1. We can get:

$$\mathcal{A}_{Evol} \approx \mathcal{P}(c \mid p) = \frac{\mathcal{P}(p \mid c) \cdot \mathcal{P}(c)}{\mathcal{P}(p)} < \mathcal{P}(p \mid c) \approx \mathcal{A}_{OSS} < 1 - (1 - \mathcal{P}(p \mid c))^n < \mathcal{A}_{AIEV}$$

Thus, under these assumptions, the accuracy of *AutoCoder-AIEV-Instruct* should be higher than that of *Magicoder-OSS-Instruct* and *Magicoder-Evol-Instruct*.

4 AutoCoder

4.1 Code Interpreter

Code Interpreter assists the model in debugging and executing code, which is essential for fully automating complex coding, scientific computations, and related tasks. Building a code interpreter requires the model to accurately identify the code blocks it needs to run. Currently, only a few models, like GPT-4 Turbo and InternLM-Chat [10], support code interpreters. However, a significant limitation of these interpreters is that they operate in a closed environment and cannot interact with external systems, preventing them from executing code that requires external package installations. AutoCoder addresses this issue by enabling the execution of `bash` commands to install necessary packages. This capability is achieved by teaching the model to run `bash` commands when appropriate. To facilitate this, we need to perform some post-processing on the *AutoCoder-AIEV-Instruct* dataset.

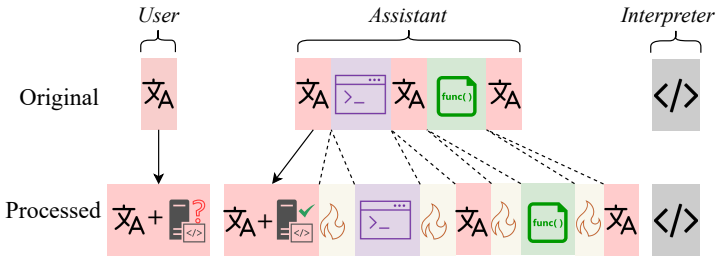

Figure 5: *AutoCoder-AIEV-Instruct* dataset post-processing. :Nature language; :Code execution request from the User; :Code execution request response from the Assistant; :Bash command; :Code block; :Special token; :Execution result.

As shown in Figure 5, for a simple single execution feedback example, the original data entry contains three parts: natural language from the **User**; natural language + `bash` command + natural language + code block + natural language from the **Assistant**; execution result from the **code interpreter**.

In the post-processing stage, we mix the natural language of the Code execution request into the User's natural language, enabling the model to correctly learn when to execute the code. Then, we mix the code execution request response into the Assistant's response, so it can generate coherent answers. Finally, we add special tokens before and after the `bash` commands and code blocks in the Assistant's original response, allowing the model to learn to correctly identify the `bash` commands and code blocks that need to be executed.

4.2 Training

We fine-tuned two base models, Deepseek-Coder 6.7B and 33B, using the *AutoCoder-AIEV-Instruct* dataset to obtain our AutoCoder 33B and AutoCoder-S 6.7B. We utilized the `AutoTokenizer` package from the `transformer` library to add four special tokens to these models to enable the Code Interpreter feature for AutoCoder. For hardware, we used 10 nodes with a total of 40 80GB A100 GPUs on a Simple Linux Utility for Resource Management (SLURM) cluster. The NVIDIA Collective Communications Library (NCCL) handled communication between GPUs. In terms of training parameters, we used the ZeRO-Stage 3 feature from the `deepspeed` library to partition model parameters, with a batch size of 8 per GPU, a gradient accumulation step of 4, a learning rate of 5e-5, and bf16 as the parameter type. The max sequence length was set to 5120 and the total epochs was set to 2. We adopted a full-parameter tuning approach to train the model.

5 Experiment

We tested AutoCoder's abilities in Python text-to-code generation, multilingual code generation, and code generation for data science questions, and compared it with the other models. To ensure a fair comparison with other models and reduce experimental randomness, we **disabled AutoCoder's external code interpreter during the tests** and used greedy sampling.

5.1 Python Text to Code Generation

We evaluated AutoCoder using two of the most commonly used code generation benchmarks: HumanEval [13] and MBPP [8]. HumanEval is widely used to test various state-of-the-art closed-source models, such as GPT-4o [27], Claude-3-Ops [7], Gemini Ultra 1.0 [14], and Llama3 400b [5]. It contains 164 code generation problems. Compared to HumanEval, MBPP has more test data, with a total of 378 test cases. Additionally, to prevent errors due to the insufficient number of test cases for each code problem in the original benchmarks, HumanEval+ and MBPP+ [24] have added more test cases to the original datasets. The Pass@1 results are shown in Table 1.

Experimental results show that AutoCoder-33B achieved a Pass@1 of 90.9% on the HumanEval benchmark, surpassing all current SOTA code LLMs. On HumanEval+, it achieved a Pass@1 of 78%, second only to GPT-4 Turbo and CodeQwen1.5-Chat. In the MBPP and MBPP+ tests, its Pass@1 was 82.5% and 70.6% respectively, ranking just below the two large closed-source models GPT-4 Turbo and Claude 3 Opus. Additionally, despite having only 6.7B parameters, AutoCoder-S still shows impressive performance. It achieved 78.7% and 72% on HumanEval and HumanEval+, respectively. In the MBPP and MBPP+ benchmarks, it achieved 79.4% and 69.8%. On MBPP+, its performance is second only to DeepSeek-Coder-instruct (33B) within the 70B parameter level.

5.2 Multilingual Code Generation

To test AutoCoder's capabilities in multilingual code generation, we used MultiPL-E benchmark [11] to evaluate its performance in six additional commonly used languages. Since MultiPL-E's official library does not support testing closed-source models, we ensured consistent experimental conditions by comparing only with well-known open-source models. The results are shown in Table 2.

The experimental results show that AutoCoder performed exceptionally well in Java, C++, and Rust, achieving 61.4%, 68.9%, and 60.8% Pass@1 respectively. In the other three languages, its performance was only surpassed by a few models such as CodeQwen1.5-Chat. This demonstrates AutoCoder's robust capabilities in multilingual code generation.

Table 1: Comparison with current SOTA Code Large language models on HumanEval(+) and MBPP(+). The results for GPT-4o, Llama3-400B, and Gemini Ultra 1.0 are sourced from the GPT-4o website [27]. The remaining measurement results are sourced from the Evalplus leaderboard [3].

Model	Size	Benchmark (Pass@1 %)			
		HumanEval	HumanEval+	MBPP	MBPP+
GPT-4o	🔒	90.2	-	-	-
GPT-4-Turbo	🔒	90.2	**86.6**	85.7	**73.3**
Claude 3 Opus	🔒	84.9	77.4	**89.4**	**73.3**
Llama3	$400B$	84.1	-	-	-
Gemini Ultra 1.0	🔒	74.4	-	-	-
OpenCodeInterpreter-CL	$70B$	76.2	70.7	73.0	61.9
CodeLlama-Instruct	$70B$	72.0	65.2	75.4	61.7
DeepSeek-Coder-instruct	$33B$	81.1	75.0	80.4	70.1
WizardCoder-V1.1	$33B$	79.9	73.2	-	-
OpenCodeInterpreter-DS	$33B$	79.3	73.8	80.2	68.5
speechless-codellama-v2.0	$34B$	77.4	72.0	73.8	61.4
OpenCodeInterpreter-CL	$13B$	77.4	73.8	70.7	59.2
starchat2-v0.1	$15B$	73.8	71.3	74.9	64.6
starcoder2-instruct-v0.1	$15B$	67.7	60.4	78.0	65.1
WizardCoder-15B-V1.0	$15B$	56.7	50.6	64.3	54.2
CodeQwen1.5-Chat	$7B$	83.5	78.7	79.4	69.0
OpenCodeInterpreter-DS	$6.7B$	77.4	72.0	76.5	66.4
Artigenz-Coder-DS	$6.7B$	75.6	72.6	80.7	69.6
DeepSeek-Coder-instruct	$6.7B$	74.4	71.3	74.9	65.6
AutoCoder-S	$6.7B$	78.7	72.0	79.4	69.8
AutoCoder	$33B$	**90.9**	78.0	82.5	70.6

Table 2: Performance (Pass@1 %) of AutoCoder on the MultiPL-E benchmark.

Model	Size	Programming Language					
		Java	JavaScript	C++	PHP	Swift	Rust
Wizard-CL	$34B$	44.9	55.3	47.2	47.2	44.3	46.2
CodeLLAMA	$34B$	40.2	41.7	41.4	40.4	35.3	38.7
CodeLLAMA-Instruct	$34B$	41.5	45.9	41.5	37	37.6	39.3
Deepseek-Coder-Instruct	$33B$	53.8	67.7	63.3	54.7	51.3	54.4
OpenCodeInterpreter-DS	$33B$	60.1	69.6	67.1	59.6	54.4	60.2
StarCoder-Base	$15B$	28.5	31.7	30.6	26.8	16.7	24.5
StarCoder	$15B$	30.2	30.8	31.6	26.1	22.7	21.8
WizardCoder-SC	$15B$	35.8	41.9	39.0	39.3	33.7	27.1
CodeLLAMA	$7B$	29.3	31.7	27.0	25.1	25.6	25.5
Magicoder-CL	$7B$	36.4	45.9	36.5	39.5	33.4	30.6
MagicoderS-CL	$7B$	42.9	57.5	44.4	47.6	44.1	40.3
CodeQwen1.5-Chat	$7B$	58.6	**75.7**	65.2	**68.9**	**58.8**	51.1
AutoCoder-S	$6.7B$	55.7	65.2	62.7	59.6	41.1	50.6
AutoCoder	$33B$	**61.4**	68.9	**68.9**	63.4	53.8	**60.8**

5.3 Code Generation for Data Science

We tested AutoCoder's ability to generate code to solve data science problems using the DS-1000 dataset. The DS-1000 dataset contains 1000 questions that require the use of seven commonly used Python data science libraries. We tested all the models using the *completion* mode in DS-1000.

As shown in Table 3 the AutoCoder's Pass@1 on Matplotlib-related questions even surpassed that of GPT-4 Turbo. Overall, it is the only model besides GPT-4 to achieve an overall Pass@1 exceeding 45%. This demonstrates AutoCoder's excellent capability to generate code for data science problems..

Table 3: Performance (Pass@1 %) of AutoCoder on the DS-1000 dataset. plt: Matplotlib, np: NumPy , Pd: Pandas, Py: PyTorch, Scp: Scipy, Sk: Sklearn , TF: TensorFlow. The result of GPT-4 Turbo `2024-04-09`, GPT-3.5 Turbo `0125` and Codex-002 are from the Offical Github of DS-1000 [6].

Model	Size	155	220	291	68	106	115	45	1000
		plt	np	Pd	Py	Scp	Sk	TF	Overall
GPT-4 Turbo	🔒	72.3	**61.8**	**42.3**	**50**	**50**	**50.4**	**53.3**	**53.9**
GPT-3.5 Turbo	🔒	65.8	32.7	30.2	36.8	39.6	40	42.2	39.4
Codex-002	🔒	57	43.1	26.5	41.8	31.8	44.8	39.3	39.2
DeepSeek-Coder-Instruct	$33B$	61.3	50.0	30.9	35.3	36.8	45.2	40.0	42.8
OpenCodeInterpreter-DS	$33B$	39.4	57.7	28.2	47.1	40.6	49.6	42.2	42.1
CodeGen-Mono	$16B$	31.7	10.9	3.40	7.00	9.00	10.8	15.2	11.7
StarCoder	$15B$	51.7	29.7	11.4	21.4	20.2	29.5	24.5	26.0
WizardCoder-SC	$15B$	55.2	33.6	16.7	26.2	24.2	24.9	26.7	29.2
CodeLlama-Python	$7B$	55.3	34.5	16.4	19.9	22.3	17.6	28.5	28.0
WizardCoder-CL	$7B$	53.5	34.4	15.2	25.7	21.0	24.5	28.9	28.4
Magicoder-CL	$7B$	54.6	34.8	19.0	24.7	25.0	22.6	28.9	29.9
MagicoderS-CL	$7B$	55.9	40.6	28.4	40.4	28.8	35.8	37.6	37.5
InCoder	$6.7B$	28.3	4.4	3.1	4.40	2.80	2.80	3.80	7.40
AutoCoder-S	$6.7B$	52.9	38.2	31.6	30.9	31.1	39.1	31.1	37.1
AutoCoder	$33B$	**72.9**	52.7	36.1	26.5	45.3	46.1	42.2	47.2

5.4 Comparison with the Base Model

To more intuitively compare the improvements AutoCoder brings to the base model, we compared it with several models trained on the same base model as AutoCoder. As shown in Figure 6, on the HumanEval, MBPP, and DS-1000 datasets, AutoCoder demonstrates a stronger improvement over the base model compared to DeepSeek-Coder-Instruct and OpenCodeInterpreter. Notably, DeepSeek-Coder-Instruct was trained with 2 billion tokens, while AutoCoder achieved better results with only 320 million tokens. This proves the effectiveness of the AIEV-INSTRUCT method.

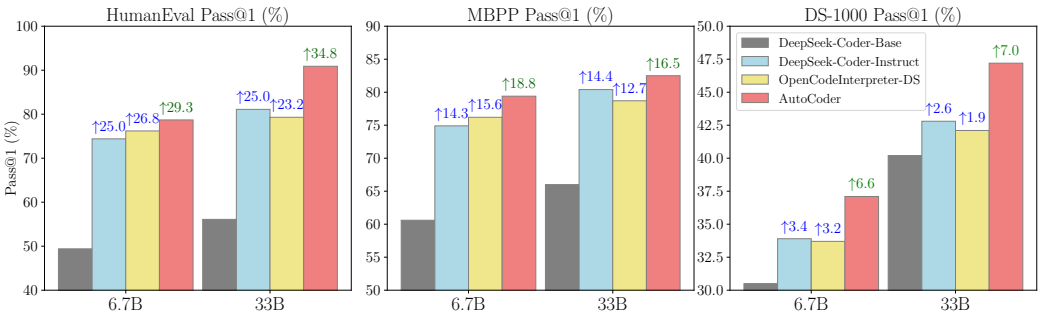

Figure 6: Comparison of AutoCoder with other models sharing the aame base model.

6 Conclusion

We propose AIEV-INSTRUCT, a novel method for creating high-quality code instruction datasets. It simulates programmers writing code and conducting unit tests through agent interactions, ensuring accuracy with execution validation. It includes both a *teaching stage* and a *self-learning stage*, reducing reliance on expensive closed-source models during the annotation process. Using the dataset generated with AIEV-INSTRUCT, we trained the AutoCoder code LLM. It exhibits excellent performance and surpass the current top models, GPT-4 Turbo and GPT-4o on the HumanEval benchmark. Furthermore, AutoCoder extends the functionality of previous code interpreters by allowing them to automatically install external packages, thus extending the applicability of the code interpreter. Overall, our work provides the community with excellent open-source code large language models and offers new insights for generating high-quality large code instruction dataset.

References

[1] Claude.ai onboarding. https://www.anthropic.com/claude.

[2] Deepseekcoder. https://deepseekcoder.github.io/.

[3] Evalplus leaderboard. https://evalplus.github.io/leaderboard.html. Accessed: 2024-05-15.

[4] Google gemini. https://gemini.google.com/.

[5] Meta AI. Meta llama 3. https://ai.meta.com/blog/meta-llama-3/, 2024.

[6] XLang AI. Ds-1000 results, 2023. URL https://github.com/xlang-ai/DS-1000/tree/main/results.

[7] Anthropic. Claude 3 family. https://www.anthropic.com/news/claude-3-family, 2024.

[8] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[9] Alessio Buscemi. A comparative study of code generation using chatgpt 3.5 across 10 programming languages. *arXiv preprint arXiv:2308.04477*, 2023.

[10] Zheng Cai, Maosong Cao, Haojiong Chen, Kai Chen, Keyu Chen, Xin Chen, Xun Chen, Zehui Chen, Zhi Chen, Pei Chu, et al. Internlm2 technical report. *arXiv preprint arXiv:2403.17297*, 2024.

[11] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, et al. Multiple: A scalable and extensible approach to benchmarking neural code generation. *arXiv preprint arXiv:2208.08227*, 2022.

[12] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology*, 15(3):1–45, 2024.

[13] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[14] DeepMind. Gemini ultra. https://deepmind.google/technologies/gemini/ultra/, 2024.

[15] Tianyu Gao, Adam Fisch, and Danqi Chen. Making pre-trained language models better few-shot learners. *arXiv preprint arXiv:2012.15723*, 2020.

[16] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y Wu, YK Li, et al. Deepseek-coder: When the large language model meets programming–the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024.

[17] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

[18] Matthew Jin, Syed Shahriar, Michele Tufano, Xin Shi, Shuai Lu, Neel Sundaresan, and Alexey Svyatkovskiy. Inferfix: End-to-end program repair with llms. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1646–1656, 2023.

[19] Majeed Kazemitabaar, Xinying Hou, Austin Henley, Barbara Jane Ericson, David Weintrop, and Tovi Grossman. How novices use llm-based code generators to solve cs1 coding tasks in a self-paced learning environment. In *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research*, pages 1–12, 2023.

[20] Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-tau Yih, Daniel Fried, Sida Wang, and Tao Yu. Ds-1000: A natural and reliable benchmark for data science code generation. In *International Conference on Machine Learning*, pages 18319–18345. PMLR, 2023.

[21] Jia Li, Ge Li, Chongyang Tao, Huangzhao Zhang, Fang Liu, and Zhi Jin. Large language model-aware in-context learning for code generation. *arXiv preprint arXiv:2310.09748*, 2023.

[22] Jia Li, Ge Li, Yunfei Zhao, Yongmin Li, Zhi Jin, Hao Zhu, Huanyu Liu, Kaibo Liu, Lecheng Wang, Zheng Fang, et al. Deveval: Evaluating code generation in practical software projects. *arXiv preprint arXiv:2401.06401*, 2024.

[23] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.

[24] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems*, 36, 2024.

[25] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. Wizardcoder: Empowering code large language models with evol-instruct. *arXiv preprint arXiv:2306.08568*, 2023.

[26] OpenAI. Chatgpt: Language models for conversational ai. https://openai.com/index/chatgpt/, 2024.

[27] OpenAI. Hello gpt-4o. https://openai.com/index/hello-gpt-4o/, 2024.

[28] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.

[29] Qwen. Codeqwen 1.5: Next generation code model. https://qwenlm.github.io/blog/codeqwen1.5/, 2024.

[30] Nikitha Rao. *Navigating Challenges with LLM-based Code Generation using Software-specific Insights*. PhD thesis, Microsoft Research, 2024.

[31] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. *arXiv preprint arXiv:2212.10560*, 2022.

[32] Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. Magicoder: Source code is all you need. *arXiv preprint arXiv:2312.02120*, 2023.

[33] Frank F Xu, Bogdan Vasilescu, and Graham Neubig. In-ide code generation from natural language: Promise and challenges. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 31(2):1–47, 2022.

[34] Tianyu Zheng, Ge Zhang, Tianhao Shen, Xueling Liu, Bill Yuchen Lin, Jie Fu, Wenhu Chen, and Xiang Yue. Opencodeinterpreter: Integrating code generation with execution and refinement. *arXiv preprint arXiv:2402.14658*, 2024.